GOLDEN RASTREAMENTO

Instalação completa do sistema

Site, captação de leads, painel (CRM, CMS e ERP) e loja virtual — do primeiro `git clone` ao checklist de saída.

Versão **2.12.0** Data **23/10/2026** Requisito **PHP 8.3 + MySQL**

Responsável **Marcelo · KDKHOST** Contato **contato@kdkhost.com.br**

1. O que a instalação faz

1. Confere PHP, extensões e Composer antes de mexer em qualquer coisa.
2. Cria o arquivo `.env` de produção (chave da aplicação, endereço, banco, fuso horário, filas e cache) com permissão `600`.
1. Instala as dependências do projeto sem as bibliotecas de desenvolvimento.
2. Prepara o banco: se ele já tiver tabelas de outro sistema, tira uma cópia de segurança e recria as tabelas do Golden a partir das migrations.
1. Grava os dados de demonstração (catálogo, páginas do site, leads de exemplo) — ou só o catálogo base, com `--sem-demo`.
1. Define o login do painel.
2. Cria o link `public/storage` para as imagens, vídeos e documentos enviados.
3. Escreve um `.htaccess` na raiz que bloqueia código-fonte, `.env` e logs, faz todo pedido entrar pela pasta `public` e, com `--php-web=`, escolhe o PHP que serve a web.
1. Liga os caches de configuração, rotas e views.

2. Abre o próprio `--url` no final e avisa se o PHP da web for outro.

Nada de credencial fica no repositório: o instalador perguntou na tela ou leu de uma variável de ambiente e gravou apenas no `.env`, que está no `.gitignore`.

---

2. Requisitos

Item	Mínimo	Como conferir
PHP	8.3	`php -v`
Extensões obrigatórias	pdo, mbstring, openssl, tokenizer, xml, fileinfo, curl, json	`php -m`
Extensões recomendadas	gd, intl, zip, exif, pdo_mysql	`php -m`
Composer	2.x	`composer --version`
MySQL	5.7 / MariaDB 10.3+	—
Apache com `mod_rewrite`	—	necessário quando o docroot é a raiz do projeto
Limite de envio	`upload_max_filesize` e `post_max_size` ≥ 40 MB	`php -i | grep upload_max`

O `gd` é o que permite ler a largura e a altura da imagem no campo de envio; sem ele o upload funciona, mas a dica de dimensão fica vazia. `intl` e `zip` são exigidos por pacotes do Composer; `exif` ajuda a validar foto de celular.

PHP 8.3 é inegociável

O `composer.json` pede `php: ^8.3` (Laravel 13). Se o servidor responder com

`Composer detected issues in your platform: Your Composer dependencies require a PHP version ">= 8.3.0". You are running 8.1.34.`

o site inteiro fica em erro 500, mesmo com banco e arquivos corretos. O ponto é que existem **dois PHPs no servidor**: o do comando `php` (linha de comando) e o que atende o navegador. Mudar um não muda o outro, e é o segundo que quebra o site.

- **cPanel:** *Software → MultiPHP Manager* → marque o domínio → *PHP Version*

→ escolha 8.3 (ou 8.4) → *Apply*. Depois confira em *MultiPHP INI Editor* os limites de `upload_max_filesize` e `post_max_size`.

- **cPanel com CloudLinux + LiteSpeed (o caso mais chato):** o *Select PHP

Version* da conta é um seletor próprio e pode parar no 8.2 mesmo com o 8.3 instalado no servidor. Nesse caso o `.htaccess` resolve, sim, desde que use o handler certo. Acrescente esta linha ao `.htaccess` da raiz do projeto (ou do `public/`, se o docroot for ele):

``` `apache AddType application/x-httpd-ea-php83 .php` ```

Foi exatamente assim que o 8.1 virou 8.3 num servidor cPanel + CloudLinux + LiteSpeed: o seletor da conta não oferecia 8.3 (`cloudlinux-selector get --interpreter php --get-supported-versions` parava em 8.2), mas o `ea-php83` estava instalado e o handler assumiu os arquivos `.php`. O instalador faz isso sozinho com `--php-web=ea-php83`.

Confira depois com um arquivo temporário dentro de `public/`:

``` `php <?php echo PHP_VERSION . ' | ' . PHP_SAPI; // salve como public/_chk.php` ```

Abra `https://seudominio/_chk.php`, confirme `8.3.x | litespeed` e **apague o arquivo** em seguida. Se aparecer `apachectl/lsapi` diferente ou a versão antiga, o handler escolhido não existe no servidor — liste os disponíveis com `ls -d /opt/cpanel/ea-php*`.

- **Extensões que costumam faltar no PHP da web:** `fileinfo` (upload de

imagem, vídeo e documento) e `intl`. No seletor do CloudLinux elas se acendem pela linha de comando, com o nome de estado por extensão — e a lista **substitui** a anterior, então informe tudo de uma vez:

``` `sh cloudlinux-selector set --json --interpreter php --version 8.3 --user SEU_USUARIO \ --extensions '{"fileinfo":"enabled","intl":"enabled","gd":"enabled","zip":"enabled","pdo_mysql":"enabled","mbstring":"enabled","bcmath":"enabled","dom":"enabled"}'` ```

- **Plesk:** *PHP Versions / Settings* → PHP 8.3.
- **VPS (Ubuntu):** `apt install php8.3-fpm php8.3-mysql php8.3-gd php8.3-intl php8.3-zip`

e aponte o `fastcgi_pass` do nginx para o 8.3.

O instalador detecta a versão do binário que você passou (`--php=`) e **para antes** de instalar qualquer coisa se ela for menor que 8.3. No final ele ainda abre o `--url` informado: se a home responder o erro de plataforma, ele avisa na tela que o PHP **da web** é outro e mostra a linha `--php-web=` que resolve.

---

3. Instalação com o instalador (recomendado)

Coloque o código no servidor (git ou upload do `.zip`), entre na pasta do projeto e rode:
```

```
GOLDEN_DB_PASSWORD='senha-do-banco' GOLDEN_ADMIN_PASSWORD='senha-do-painel' \
  bash deploy/instalador-golden.sh \
    --url=https://seudominio.com.br \
    --nome=nome_do_banco \
    --usuario=usuario_do_banco
```

Passar as senhas por variável de ambiente evita que elas fiquem no histórico do shell; sem as variáveis, o instalador pede na tela sem mostrar o que foi digitado.

Opções do instalador

Opção	Para que serve
`--url=`	endereço público do site, com `https://` (obrigatório)
`--banco=mysql|sqlite`	motor do banco; padrão `mysql`
`--host=`, `--porta=`	servidor MySQL; padrão `127.0.0.1:3306`
`--nome=`	nome do banco (no SQLite, o arquivo)
`--usuario=`, `--senha=`	credenciais do banco
`--criar-banco`	cria o banco se não existir (precisa de privilégio `CREATE`)
`--admin-email=`, `--admin-nome=`, `--admin-senha=`	login do painel
`--sem-demo`	não grava os dados de demonstração
`--limpar-banco`	apaga todas as tabelas antes de recriar (sem a flag, ele pergunta)
`--sem-backup`	não tira cópia do banco antes de limpar
`--dev`	`APP_ENV=local` e `APP_DEBUG=true` (nunca em produção)
`--sem-htaccess`	não escreve o `.htaccess` de proteção da raiz
`--sem-cache`	não gera cache de configuração, rotas e views
`--php=`, `--composer=`	força o binário quando há mais de uma versão no servidor
`--php-web=ea-php83`	grava no `.htaccess` o handler do PHP que serve a web (CloudLinux/LiteSpeed)
`--help`	tela de ajuda completa

Exemplos prontos

cPanel, reaproveitando um banco que já existe no plano

```
GOLDEN_DB_PASSWORD='...' GOLDEN_ADMIN_PASSWORD='...' \
  bash deploy/instalador-golden.sh \
    --url=https://seudominio.com.br \
    --php=/usr/local/bin/ea-php83 --php-web=ea-php83 \
    --nome=usuario_do_banco --usuario=usuario_do_banco \
    --limpar-banco
```

VPS nova, criando o banco

```
bash deploy/instalador-golden.sh --url=https://painel.exemplo.com.br \
  --nome=golden --usuario=golden --criar-banco
```

Teste local, sem MySQL

```
bash deploy/instalador-golden.sh --url=http://localhost:8000 \
  --banco=sqlite --dev --sem-htaccess
```

O que aparece na tela

Cada etapa imprime uma linha com ✓. Se algo falhar, o instalador para, mostra o comando que falhou e as últimas 25 linhas do log (`storage/logs/instalador-*.log`), que guarda a saída completa. No fim ele imprime o resumo com endereço, painel, banco, ambiente e caminho do log.

Limpar um banco que já tem outro sistema

O instalador conta as tabelas antes de encostar. Havendo tabelas, ele:

1. faz `mysqldump` comprimido para a pasta **acima** do projeto — fora do que o

navegador serve — com permissão `600`;

1. pede confirmação explícita (digitar `SIM`), ou exige a flag `--limpar-banco`

quando roda sem terminal;

1. só então roda `migrate:fresh`.

Para restaurar a cópia depois:

```
gunzip < ../backup-banco-nome-20261023-1130.sql.gz | mysql -u USUARIO -p NOME_DO_BANCO
```

---

4. Instalação manual, passo a passo

Serve para qualquer servidor e é o roteiro usado quando o instalador não pode rodar (por exemplo, hospedagem sem SSH).

4.1 Subir o código

- **Com git:** `git clone https://github.com/kdkhost/GOLDENRASTREAMENTO.git public_html`
- **Sem git:** baixe o `.zip` do repositório e extraia na pasta do domínio.

Confirme que `artisan`, `app/`, `public/` e `composer.json` estão na raiz da pasta extraída.

4.2 Criar o banco (cPanel)

1. *MySQL® Databases* → criar banco; a hospedagem costuma prefixar com o nome

da conta, ex.: `seuusuario_cms`. Anote o nome completo.

1. Criar usuário e **senha forte**; anote os dois.
2. Adicionar o usuário ao banco com **ALL PRIVILEGES**.
3. *phpMyAdmin* → conferir que o banco aparece e está vazio.

Se aparecer `Access denied for user`, faltou ligar o usuário ao banco.

4.3 Criar o `.env`

```
cp .env.example .env
php artisan key:generate
chmod 600 .env
```

Ajuste, no mínimo:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://seudominio.com.br

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=seuusuario_cms
DB_USERNAME=seuusuario_cms
DB_PASSWORD=senha-do-banco
```

Veja a referência completa de todas as chaves na seção 7.

4.4 Dependências

```
composer install --no-dev --prefer-dist --optimize-autoloader
```

Sem Composer no PATH, baixe o arquivo único e use pelo PHP:

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php --install-dir=$HOME/bin --filename=composer.phar
php $HOME/bin/composer.phar install --no-dev --prefer-dist --optimize-autoloader
```

4.5 Banco de dados

```
php artisan migrate --force           # só as tabelas
php artisan migrate --force --seed    # com catálogo, páginas e demonstração
```

Para começar do zero em um banco que já tinha outra coisa:

```
php artisan migrate:fresh --seed --force
```

4.6 Imagens, vídeos e documentos

```
php artisan storage:link
```

Cria `public/storage` apontando para `storage/app/public`. Se o link não puder ser criado (algumas hospedagens bloqueiam o `symlink`), faça manualmente:

```
ln -s ../storage public/storage
```

Permissões: pastas `755`, arquivos `644`, e `storage/` + `bootstrap/cache/` precisam ser graváveis pelo usuário do webserver.

4.7 Entrega pelo docroot

Quando o domínio aponta para a **raiz do projeto** (caso típico do cPanel, `public_html`), use o `.htaccess` gerado pelo instalador — ele:

- desliga o índice de diretório (`Options -Indexes`), o que impede a listagem do

código-fonte;

- bloqueia `.env`, `.env.*`, `.git*`, `artisan`, `composer.*`, `package.json`,

`vite.config.js`, `phpunit.xml`, `error_log` e qualquer `.log`;

- bloqueia as pastas `app`, `bootstrap`, `config`, `database`, `resources`,

`routes` e `tests`;

- reescreve todo pedido para dentro de `public/`.

`storage` **não** entra na lista de bloqueio: o que o público alcança é o link `public/storage`, por onde saem as imagens, vídeos e documentos enviados pelo painel. Bloquear o caminho `/storage` na raiz devolve 403 em todo arquivo enviado, e o site fica com as imagens quebradas. A privateidade do `storage/` de verdade vem da reescrita (nada alcança `storage/logs`, `storage/framework` ou `storage/app/private`) e do bloqueio de `.log`.

O teste de barreira, já com o site no ar:

```
curl -I https://seudominio.com.br/.env          # 403
curl -I https://seudominio.com.br/app/          # 403
curl -I https://seudominio.com.br/storage/logs/laravel.log   # 403
curl -I https://seudominio.com.br/              # 200
curl -I https://seudominio.com.br/loja          # 200
```

E o teste do que o painel envia, que é o que costuma sobrar depois de mexer no `.htaccess`: envie uma imagem pelo painel e abra a URL devolvida no JSON de `/admin/uploads`. Tem de ser `200` com o `Content-Type` do arquivo — se der 403, a regra de bloqueio está alcançando `public/storage`.

Quando o docroot é a pasta `public/` (VPS com nginx, ou subdomínio apontando para `public`), o `.htaccess` da raiz nem é lido — e a proteção já existe por construção.

4.8 Caches de produção

```
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Depois de qualquer mudança em `.env`, `config/`, `routes/` ou `resources/views/` rode os três de novo (ou `php artisan optimize:clear` e recarregue).

4.9 Primeiro acesso

- Site: `https://seudominio.com.br`
- Painel: `https://seudominio.com.br/admin` — login definido na instalação.
- Troque a senha padrão assim que entrar, e remova o usuário de demonstração

(`vendedor@golden.com.br`) se não for usar.

---

5. Instalação no cPanel, em detalhe

1. *MultiPHP Manager* → PHP **8.3** para o domínio. Se o plano usar CloudLinux,

confira também o *Select PHP Version* da conta; quando ele não chega no 8.3, pule essa etapa e deixe o `.htaccess` escolher o handler (`--php-web=ea-php83`, seção 2).

1. *MySQL® Databases* → banco + usuário + privilégios (seção 4.2).
2. *Git™ Version Control* (se houver): *Clone Repository* com a URL pública do

repositório e o branch `main`, caminho `public_html`. Não use credencial na URL: o repositório é público. Sem o Git™, envie o `.zip` pelo *Gerenciador de Arquivos* e extraia.

1. *Terminal* (ou SSH com chave) → na pasta do projeto:

`git pull`, depois o instalador:

``bash GOLDEN_DB_PASSWORD='...' GOLDEN_ADMIN_PASSWORD='...' \ bash deploy/instalador-golden.sh \ --url=https://seudominio.com.br \ --php="$(command -v ea-php83)" --php-web=ea-php83 \ --nome=usuario_mysql --usuario=usuario_mysql --limpar-banco``

1. *Gerenciador de Arquivos* → confira as permissões: `.env` em `600`,

pastas `755`.

1. *MultiPHP INI Editor* → `upload_max_filesize=40M`, `post_max_size=45M`,

`max_execution_time=120` (o campo de vídeo aceita até 40 MB).

1. Abra o site e faça o teste de barreira da seção 4.7.
2. *SSL/TLS → Let's Encrypt* (ou o auto-SSL da hospedagem) ativo, e

Forçar HTTPS ligado.

Para outros agentes ou para usar de outra máquina, a mesma conta pode ser acessada por chave SSH: gere `ssh-keygen -t ed25519 -f ~/.ssh/golden_deploy -N ""`, cole a `.pub` em *Segurança → Chaves SSH / Access Hash*, e crie um apelido no `~/.ssh/config`. **Nunca** coloque senha, chave ou `.env` dentro do repositório.

---

6. Atualização depois da primeira instalação

```
bash deploy/atualiza-golden.sh                  # ramo main
bash deploy/atualiza-golden.sh --ramo=main --php=/usr/local/bin/ea-php83
```

Ele faz `git pull --ff-only`, `composer install --no-dev`, `migrate --force`, `storage:link` e reativa todos os caches. Se uma migration nova existir, ela é aplicada sem tocar nos dados.

Quando mexer em `.env` à mão:

```
php artisan config:clear && php artisan config:cache
```

---

7. Referência do `.env`

Chave	Produção	Observações
`APP_NAME`	`Golden Rastreamento`	nome em e-mails e no painel
`APP_ENV`	`production`	nunca `local` em servidor
`APP_DEBUG`	`false`	`true` vaza pilha de erro para o visitante
`APP_URL`	`https://seudominio.com.br`	usado em links de e-mail e assets
`APP_KEY`	gerado por `key:generate`	se trocar, sessões e dados criptografados somem
`APP_TIMEZONE`	`America/Sao_Paulo`	datas de pedidos e leads
`APP_LOCALE` / `APP_FALLBACK_LOCALE`	`pt_BR`	textos e moeda
`DB_CONNECTION`	`mysql`	`sqlite` apenas em teste
`DB_HOST`, `DB_PORT`	`127.0.0.1`, `3306`	no cPanel costuma ser `localhost`
`DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`	do painel	nunca no git
`SESSION_DRIVER`	`file`	sessões não geram consulta ao banco
`CACHE_STORE`	`file` (ou `redis`)	
`QUEUE_CONNECTION`	`database`	exige `php artisan queue:work`
`FILESYSTEM_DISK`	`public`	onde os envios vão parar
`LOG_CHANNEL` / `LOG_LEVEL`	`stack` / `warning`	log em `storage/logs`
`MAIL_MAILER`	`smtp`	use `log` enquanto não tiver credencial
`MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_ENCRYPTION`	do seu servidor de correio	o **código de e-mail está pronto no projeto**; falta só a credencial do servidor
`MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`	`comercial@goldenrastreamento.com.br`	remetente dos orçamentos e avisos
`GOLDEN_WHATSAPP`	`5519997101483`	botão de WhatsApp do site

Chave	Produção	Observações
`GOLDEN_EMAIL`	endereço comercial	aparece no site e nos avisos
`GOLDEN_TRACKING_ENABLED`	`0` ou `1`	liga a consulta de rastreio

Sobre os e-mails: os modelos, o envio de orçamento e as notificações de pedido já existem. Trocar `MAIL_MAILER=log` para `smtp` e preencher as cinco chaves é tudo que falta para eles saírem de verdade. Em `log`, a mensagem fica escrita em `storage/logs/laravel.log` — útil para conferir o conteúdo antes de liberar o SMTP.

---

8. Loja virtual

A loja (catálogo, carrinho, checkout e pedidos no painel) vem ligada. Para desligar a vitrine sem desligar o resto, use o painel em *ERP → Configurações da loja* (chaves `loja_ativa` e `loja_livre`); o site some do menu e o `/loja` responde 404, mas os pedidos já registrados continuam no painel.

Os campos de envio do painel (imagem do produto, vídeo do produto e documentos do pedido) aceitam arrastar-e-soltar, mostram preview, progresso real, tempo decorrido e a regra do campo (formatos, tamanho máximo e dimensão ideal). As regras moram em `config/envios.php`:

Campo	Formatos	Limite	Pasta no disco
`produto_imagem`	JPG, JPEG, PNG, WEBP	4 MB	`produtos/`
`produto_video`	MP4, WEBM	40 MB	`produtos/video/`
`pedido_documento`	PDF, JPG, JPEG, PNG	10 MB cada, até 12 por pedido	`pedidos/`

O servidor confere a extensão **do conteúdo** do arquivo, não o nome que o navegador mandou, e recusa o envio se o tipo não estiver entre os formatos do campo. Por isso os limites de `upload_max_filesize` / `post_max_size` da seção 2 importam: arquivo maior que isso chega cortado e o campo avisa o limite do servidor.

---

9. Backup e restauração

```
# banco
mysqldump --single-transaction -u USUARIO -p NOME_DO_BANCO | gzip > banco-$(date
+%F).sql.gz
# arquivos enviados
tar czf storage-$(date +%F).tar.gz storage/app/public
```

Restaurar:

```
gunzip < banco-2026-10-23.sql.gz | mysql -u USUARIO -p NOME_DO_BANCO
tar xzf storage-2026-10-23.tar.gz
php artisan storage:link
```

Guarde o `.env` fora do repositório (ele tem a `APP_KEY` e as credenciais): sem ele, os dados criptografados e as sessões não voltam.

---

10. Problemas comuns

Sintoma	Causa	Solução
Erro 500 com `Your Composer dependencies require a PHP version ">= 8.3.0"`	o PHP **da web** é antigo, mesmo com o `php -v` do SSH mostrando 8.3	MultiPHP Manager → 8.3; se o seletor da conta não chegar lá, `--php-web=ea-php83` / a linha `AddType` no `.htaccess` (seção 2)
Site mostra a lista de arquivos e pastas	docroot na raiz sem `.htaccess`, e `Options Indexes` ligado	rode o instalador (ou copie o `.htaccess` da seção 4.7)
`Route [admin.xxx] not defined` ao salvar	cache de rotas velho	`php artisan route:clear && php artisan route:cache`
Página quebra depois de editar um `.php` em `config/`	cache de configuração	`php artisan config:cache`
Imagem enviada não aparece	`public/storage` inexistente	`php artisan storage:link`
Imagem enviada existe no servidor e a URL devolve **403**	o `.htaccess` da raiz bloqueia o caminho `/storage`	tire `storage` da regra de pastas de código-fonte (seção 4.7)
Upload falha com erro em `getimagesize` / `finfo`	`fileinfo` desligada no PHP da web	ative a extensão no seletor de PHP (seção 2)
Upload falha no meio com "passou do limite do servidor"	`post_max_size` menor que o campo	ajuste no painel de PHP (40 MB para vídeo + margem)
`could not find driver`	`pdo_mysql` desligado	ative a extensão no painel de PHP
`500` com `Permission denied` em `storage/`	permissões	`chmod -R 775 storage bootstrap/cache`
`Access denied for user`	usuário sem privilégio no banco	ligar usuário ao banco no painel MySQL
E-mail não sai	`MAIL_MAILER=log` ou credencial errada	conferir `storage/logs/laravel.log`; depois configurar SMTP
Página do site antiga depois de publicar	`opcache` do servidor	`php artisan optimize:clear`; em cPanel, reinicie o PHP

Os logs ficam em `storage/logs/laravel.log`; no cPanel também existe `public/error_log`, que o `.htaccess` bloqueia para visitantes, e o erro real continua visível para você pelo painel ou SSH.

---

11. Segurança

- `.env` com `chmod 600` e fora do git; `APP_DEBUG=false` em produção.
- Senha do painel forte e única; sem `golden123` em servidor público.
- Repositório público: nenhuma credencial, chave SSH ou cópia de banco entra nele.
- O `.htaccess` bloqueia código, segredos e logs; a entrega real é sempre pela

pasta `public/`.

- Arquivo enviado vira nome legível + sufixo aleatório, na pasta do próprio

campo; a extensão é a do conteúdo, então um script batizado de `.jpg` é recusado em vez de virar `.php` executável.

- O destino do envio (`campo`) decide a pasta; o navegador nunca escolhe caminho

no disco, e o descarte só apaga arquivo que comece na pasta do próprio campo.

- Backups de banco ficam fora do docroot, com `600`.

---

12. Checklist depois de instalar

- [ ] `https://seudominio.com.br` abre e o menu funciona
- [ ] PHP da web conferido (`_chk.php` temporário, ou o aviso final do instalador)
- [ ] `/loja` lista produtos; página de produto mostra imagem e vídeo
- [ ] Checkout registra pedido e ele aparece no painel
- [ ] `/admin` abre com o login criado na instalação
- [ ] Campo de envio: arrastar imagem, vídeo e documento mostra preview e progresso
- [ ] Arquivo enviado pelo painel abre pela URL de `/storage/...` (200, não 403)
- [ ] Teste de barreira: `.env`, `app/` e `storage/logs/laravel.log` respondem 403
- [ ] Configurações do painel com WhatsApp, telefone, e-mail e endereço corretos
- [ ] SMTP preenchido (ou `log`, assumido de propósito)
- [ ] Crons: `php artisan schedule:run` a cada minuto, se usar lembretes
- [ ] Backup automático de banco e `storage/app/public`

Documento gerado de **docs/INSTALACAO.md** — a fonte é a mesma para este PDF e para o README do repositório. Nenhuma credencial aparece aqui: senha de banco, de painel e **APP_KEY** ficam apenas no **.env** do servidor.